

Experiment No - 1

Title:

Data Wrangling, I:

**Perform the following operations using Python on any open-source dataset (e.g., data.csv)
Import all the required Python Libraries.**

- 1. Locate open-source data from the web (e.g. <https://www.kaggle.com>).**
- 2. Provide a clear description of the data and its source (i.e., URL of the web site).**
- 3. Load the Dataset into the pandas data frame.**
- 4. Data Pre-processing: check for missing values in the data using pandas `isnull()`, `describe()`**
- 5. Data Formatting and Data Normalization: Summarize the types of variables by checking the data types (i.e., character, numeric, integer, factor, and logical) of the variables in the data set. If variables are not in the correct data type, apply proper type conversions.**
- 6. Turn Categorical variables into quantitative variables in python.**

Objectives

1. Locate, describe, and load open-source datasets.
2. Perform data pre-processing to handle missing values and summarize data.
3. Format and normalize variables, ensuring proper data types for analysis.
4. Develop foundational skills in data wrangling using Python.

Theory:

Python Libraries in Data Science

1) NumPy (Numerical Python)

Purpose:

NumPy is a powerful library for numerical computing in Python. It is the foundation for most other scientific computing libraries and is particularly suited for handling large multidimensional arrays and matrices.

Key Features:

- **ndarray**: The core object in NumPy, representing an N-dimensional array.
- **Vectorized operations**: Perform element-wise operations on arrays without the need for explicit loops.
- **Mathematical functions**: Includes a wide range of mathematical functions like linear algebra, statistics, Fourier transforms, etc.
- **Random sampling**: Functions for random number generation.

Common Uses:

- Efficient numerical operations and manipulation of arrays
- Mathematical and statistical operations.
- Handling large datasets in machine learning and scientific computing.

2) Pandas

Purpose:

Pandas is a data manipulation and analysis library built on top of NumPy.

It provides two primary data structures: Series (1D) and DataFrame (2D), which are powerful tools for working with structured data.

Key Features:

- **DataFrame**: A 2D labeled data structure, similar to a table, for handling heterogeneous data.
- **Series**: A 1D labeled array, often used for representing a single column in a DataFrame.
- **Data manipulation**: Tools for filtering, reshaping, merging, and grouping data.
- **Handling missing data**: Easily handle missing or NaN values with built-in functions.
- **File I/O**: Read/write data to and from a variety of file formats like CSV, Excel, SQL, JSON, etc.

Common Uses:

- Data wrangling: cleaning, transforming, and preparing data for analysis
- Exploratory data analysis (EDA) with summary statistics.
- Merging, joining, and reshaping datasets for analysis or visualization.

Data Wrangling

Data wrangling, also known as **data cleaning** or **data munging**, is the process of transforming and organizing raw data into a structured and usable format for analysis. It involves identifying and handling inconsistencies, missing values, and errors in data to ensure it is clean, accurate, and ready for processing.

Need for Data Wrangling

Data wrangling is a critical process in data analysis and preparation because raw data often comes in an unstructured, incomplete, or inconsistent format. Without proper wrangling, such data can lead to incorrect insights or models.

Below are the primary reasons why data wrangling is essential:

Key Steps in Data Wrangling

1. Collecting Data:

- o Importing data from various sources like databases, spreadsheets, APIs, or files.

2. Understanding the Data:

- o Inspecting the structure, content, and quality of the data.
- o Understanding column types, unique values, and relationships between variables.

3. Cleaning the Data:

- o Removing or filling missing values.
- o Correcting errors or inconsistencies in the data (e.g., fixing typos, removing duplicates).
- o Standardizing data formats (e.g., date formats, categorical values).

4. Transforming the Data:

- o Converting data types (e.g., numeric to categorical or vice versa).
- o Scaling or normalizing numerical data for consistency.
- o Splitting, combining, or aggregating data columns as needed.

5. Validating the Data:

- o Ensuring the cleaned data aligns with the intended structure and is free of errors.
- o Verifying that the data transformations have preserved its integrity.

6. Storing or Exporting the Data:

- o Saving the cleaned data into a format or system for further analysis or machine learning.

Why is Data Wrangling Important?

1. Improves Data Quality: Clean and structured data reduces errors in analysis or models.
2. Ensures Consistency: Helps maintain consistent formats across datasets.

3. Facilitates Analysis: Makes data easier to explore and derive insights from.
4. Supports Decision-Making: High-quality data leads to better-informed decisions.

Algorithm:

1. Locate Open-Source Data from the Web For this task, we use the “_____ .csv” dataset provided. While the file has been uploaded, a similar dataset can often be found on platforms like Kaggle.

2. **Description of the Data and Source Dataset Description:** The “_____ .csv” file appears to be a dataset containing _____-related data, potentially including attributes like _____, _____(Column names for your dataset).

Source URL: Uploaded file. If this dataset is derived from an online source, be sure to document its origin.

3. Load the Dataset into a Pandas DataFrame

We will load the dataset into a pandas DataFrame for analysis.

```
import pandas as pd

# Load the dataset into a DataFrame
dataset=pd.read_csv("_____.csv")

dataset

# Display the first few rows of the dataset
print(df.head())
```

4.Data Pre-processing

Data preprocessing is the process of preparing raw data for analysis by cleaning, transforming, and structuring it into a usable format. This is an essential step in data science to ensure that the data is accurate, consistent, and ready for analysis or modeling.

Check for Missing Values: Use pandas to identify any missing data in the dataset.

```
# Check for missing values
missing_values = df.isnull().sum()

print("Missing Values:\n", missing_values)
```

Provide a summary of the dataset

```
print("\nDataset Description:\n", df.describe(include='all'))
```

5. Data Formatting and Normalization

Data Transformation

Data transformation involves converting data into a consistent and structured format to make it suitable for analysis. This process is essential for preparing the employee dataset, ensuring that all data is clean, meaningful, and aligned with the intended purpose.

Summarize Variable Types: Check the data types of all variables to ensure they are appropriate.

#Check data types

```
print("Data Types:\n", df.dtypes)
```

Common Transformations Applied:

1. **Converting Data Types:** (Give example for conversion you perform)
 - o Columns like ID may be numeric in the raw dataset but should be treated as strings for consistency since IDs are not used for mathematical operations.
 - o Date columns, such as HireDate, need to be converted to datetime format for temporal analysis (e.g., calculating tenure).
2. **Handling Missing Values:**
 - o Missing data in columns can be filled with the mean, median, or an estimated value, depending on the context.
 - Example: (Give one example using fillna method)

Data Normalization

Normalization scales numeric data into a consistent range (e.g., 0 to 1) without distorting differences in values.

It is particularly useful for datasets with varying scales of numeric features, such as Salary and Age.

Why Normalize?

- Ensures that no single feature dominates others due to its scale.
- Improves performance and convergence of machine-learning models.

- Enhances interpretability of data.

Normalization Techniques Applied:

1. Min-Max Scaling:

o Min-max scaling is similar to z-score normalization in that it will replace every value in a column with a new value using a formula. It rescales numeric data to a fixed range, typically [0, 1]. In this case, that formula is:

$$y = (x - x_{\min}) / (x_{\max} - x_{\min})$$

Where,

- y is our new value
- x is the original cell value
- x_min is the minimum value of the column
- x_max is the maximum value of the column

Example:

```
from sklearn.preprocessing import MinMaxScaler
```

```
scaler = MinMaxScaler()
```

```
df[['petal_length']] = scaler.fit_transform(df[['petal_length']])
```

2. Z-Score Normalization (Standardization):

- o Scales data based on mean and standard deviation, ensuring a mean of 0 and standard deviation of 1.
- o Useful when the data follows a normal distribution.

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
df[['sepal_length']] = scaler.fit_transform(df[['Sepal_length']])
```

By applying these transformations and normalizations, the “iris” dataset becomes clean, consistent, and ready for advanced analysis, modeling, or visualization tasks.

Label encoding:

Label Encoding refers to converting the labels into a numeric form so as to convert them into the machine-readable form. It is an important preprocessing step for the structured dataset in supervised learning.

Algorithm:

Step 1: Import pandas and sklearn library for preprocessing

Step 2: Load the iris dataset in dataframe object df

Step 3: Observe the unique values for the Species column.

Step 4: define label_encoder object knows how to understand word labels.

Step 5: Encode labels in column 'species'.

Step 6: Observe the unique values for the Species column.

Conclusion

In this way we learned to clean, preprocess, and format datasets using Python, gaining essential data-wrangling skills.

Group A
ASSIGNMENT NO: 02

Aim:

Write a code in python using any open source platform for Data Wrangling -II

Problem Statement:

Data Wrangling, II

Create an “Academic performance” dataset of students and perform the following operations using Python.

1. Scan all variables for missing values and inconsistencies. If there are missing values and/or inconsistencies, use any of the suitable techniques to deal with them.
2. Scan all numeric variables for outliers. If there are outliers, use any of the suitable techniques to deal with them.
3. Apply data transformations on at least one of the variables. The purpose of this transformation should be one of the following reasons: to change the scale for better understanding of the variable, to convert a non-linear relation into a linear one, or to decrease the skewness and convert the distribution into a normal distribution. Reason and document your approach properly.

Objective:

To perform the data wrangling operation using Python on any open source dataset(student_pprformance.csv)

Prerequisite:

Basic concept of Pandas and numpy library. 2. Concept of Data Preprocessing, Data Formatting, Data Normalization and Data Cleaning

Theory:

- **About Dataset:**

Student Performance Data was obtained in a survey of students.

Dataset Contains Features like

- Roll No: Roll No of students

- Nationality: Contains Nationality of students
- WT: Marks Scored in Web Technology Subject
- DSBDA: Marks Scored in Data Science and Big Data Analytics Subject
- AI: Marks Scored in Artificial Intelligence Subject
- Average: Average marks scored by students for WT, DSBDA and AI
- Grade: Grade of Student (A, B, C, D)
- PG: Whether he is eligible for POSTGRADUATE or not (Yes/No)

Pandas Shape Function

The shape property is used to get a tuple representing the dimensionality of the Pandas DataFrame.

```
: student.shape
: (98, 8)
```

Identification and Handling of Null Values

Missing Data can occur when no information is provided for one or more items or for a whole unit.

Missing Data is a very big problem in real-life scenarios. Missing Data can also refer to as NA (Not Available) values in pandas. In DataFrame sometimes many datasets simply arrive with missing data, either because it exists and was not collected or it never existed. For Example, Suppose different users being surveyed may choose not to share their income, some users may choose not to share the address in this way many datasets went missing.

In Pandas missing data is represented by two value:

1. None: None is a Python singleton object that is often used for missing data in Python code.
2. NaN : NaN (an acronym for Not a Number), is a special floating-point value recognized by all systems that use the standard IEEE floating-point representation.

Pandas treat None and NaN as essentially interchangeable for indicating missing or null values. To facilitate this convention, there are several useful functions for detecting, removing, and replacing null values in Pandas DataFrame

- isnull()
- notnull()
- dropna()
- fillna()

- `replace()`

1. Checking for missing values using `isnull()` and `notnull()`

- Checking for missing values using `isnull()`

In order to check null values in Pandas DataFrame, `isnull()` function is used. This function return dataframe of Boolean values which are True for NaN values.

Data transformation:

Data transformation is the process of converting data from one format, such as a database file, XML document or Excel spreadsheet, into another.

Transformations typically involve converting a raw data source into a cleansed, validated and ready-to-use format.

Outliers:

When going through the process of data analysis, outliers can cause anomalies in the results obtained. This means that they require some special attention and, in some cases, will need to be removed in order to analyze data effectively.

There are two main reasons why giving outliers special attention is a necessary aspect of the data analytics process:

1. Outliers may have a negative effect on the result of an analysis
2. Outliers—or their behavior—may be the information that a data analyst requires from the analysis

some of the more common causes of outliers in datasets:

- Human error while manually entering data, such as a typo
- Intentional errors, such as dummy outliers included in a dataset to test detection methods
- Sampling errors that arise from extracting or mixing data from inaccurate or various sources
- Data processing errors that arise from data manipulation, or unintended mutations of a dataset
- Measurement errors as a result of instrumental error
- Experimental errors, from the data extraction process or experiment planning or execution
- Natural outliers which occur “naturally” in the dataset, as opposed to being the result of an error otherwise listed. These naturally-occurring errors are known as novelties

With small datasets, it can be easy to spot outliers manually (for example, with a set of data being 28, 26, 21, 24, 78, 22, 21, 25, -10, 22, 27 here we can see that 78 and -10 are the outlier) but when it comes to large datasets or big data, other tools are required.

Techniques to remove outliers

1. Using Scatter Plot
2. Using Box Plot
3. Using Z-Score
4. IQR - Inter Quartile Range

A) Creating Scatter Plots

With Pyplot, you can use the `scatter()` function to draw a scatter plot.

The `scatter()` function plots one dot for each observation. It needs two arrays of the same length, one for the values of the x-axis, and one for values on the y-axis:

```
plt.scatter(range(len(data)),y=data)
```

```
plt.show()
```

b) Identifying outliers with box plots

Visualizing data as a box plot makes it very easy to spot outliers. A box plot will show the “box” which indicates the interquartile range (from the lower quartile to the upper quartile, with the middle indicating the median data value) and any outliers will be shown outside of the “whiskers” of the plot, each side representing the minimum and maximum values of the dataset, respectively. If the box skews closer to the maximum whisker, the prominent outlier would be the minimum value. Likewise, if the box skews closer to the minimum-valued whisker, the prominent outlier would then be the maximum value.

```
plt.boxplot(x=data)
```

```
plt.show()
```

c) Using z-score

A z-score measures exactly how many standard deviations above or below the mean a data point is. Here's the formula for calculating a z-score: where μ is mean and σ is standard deviation

$$z = \frac{x - \mu}{\sigma}$$

```
outliers = []
def detect_outliers(x):
    threshold = 1
    mean = np.mean(x)
    std = np.std(x)
    for i in x:
        z_score = (i-mean)/std
        if np.abs(z_score) > threshold:
            outliers.append(i)
    return outliers
```

```
my_outlier = detect_outliers(data)
my_outlier
```

[78, -10]

d) IQR

The interquartile range defines the difference between the third and the first quartile. Quartiles are the partitioned values that divide the whole series into 4 equal parts. So, there are 3 quartiles. First Quartile is denoted by Q1 known as the lower quartile, the second Quartile is denoted by Q2 and the third Quartile is denoted by Q3 known as the upper quartile. Therefore, the interquartile range is equal to the upper quartile minus lower quartile.

Steps to find Outliers using IQR

1. Sort The data
2. Calculate Q1(1st Quartile 25%) and Q3(3rd Quartile 75%)
3. Calculate IQR(q3-q1)
4. Calculate lower bound
5. Calculate upper bound

Algorithm:

Step 1 : Import pandas and numpy in order to check missing values in Pandas

DataFrame

```
import pandas as pd
import numpy as np
import warnings
warnings.filterwarnings('ignore')
```

Step 2: Load the Dataset into the pandas data frame student

```
student=pd.read_csv("____.csv ")
```

Step 3: Display the data frame

```
student
```

	Roll No	Nationality	WT	DSBDA	AI	Average	Grade	PG
0	1.0	India	72.0	72.0	74.0	72.666667	C	Yes
1	2.0	Japan	69.0	90.0	NaN	53.000000	B	No
2	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
3	4.0	China	47.0	NaN	44.0	30.333333	D	Yes
4	5.0	India	76.0	78.0	NaN	51.333333	C	No
...	...	...	...	...	...	...	...	...
94	95.0	USA	79.0	NaN	NaN	26.333333	A	No
95	96.0	Japan	78.0	81.0	82.0	80.333333	A	No
96	97.0	USA	65.0	66.0	62.0	64.333333	B	No
97	98.0	Russia	63.0	72.0	70.0	68.333333	B	Yes
98	99.0	India	58.0	67.0	NaN	41.666667	B	No

99 rows × 8 columns

Step 4: Use isnull() function to check null values in the dataset.

```
student.isnull().sum()
```

Step 5: To create a series true for NaN values for specific columns. for example

math score in dataset and display data with only math score as NaN

```
series = pd.isnull(student["DSBDA"])
```

```
student[series]
```

Step 6: Drop the rows which contain any value as null values. So Dataset size reduced as 80 rows × 8

columns

```
new_student = student.dropna(axis = 0, how = 'any')  
new_student
```

OR

Find median of DSBDA by using

```
student["DSBDA"].median()
```

Replace null values of student by median of it.

```
student["DSBDA"] = student["DSBDA"].replace(student["DSBDA"].median())
```

Similarly we replace “AI” null values by it’s mode and WT values by it’s mean as

```
student["AI"] = student["AI"].replace(np.nan,student["AI"].mode()[0])  
student["WT"] = student["WT"].replace(np.nan,student["WT"].mean())
```

Pandas DataFrame ffill() Method

Pandas dataframe.ffill() function is used to fill the missing value in the dataframe. Replace NULL values with the value from the previous row:

‘ffill’ stands for ‘forward fill’ and will propagate the last valid observation forward.

Also we replace null value of Average by it’s mean and PG values by as ffill()

```
student["Average"] = student["Average"].fillna(student["Average"].mean())  
student["PG"] = student["PG"].fillna(method='ffill')
```

Step 7: Data transformation : Here we convert categorical data of student[‘PG’] which contains Yes/No information into 1 for Yes and 0 for No, using Label Encoding or One Hot Encoding.

Label Encoding is a technique that is used to convert categorical columns into numerical ones so that they can be fitted by machine learning models which only take numerical data.

```
from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()
```

```
student['PG'] = le.fit_transform(student['PG'])
```

```
student
```

	Roll No	Nationality	WT	DSBDA	AI	Average	Grade	PG
0	1	India	72.0	72.0	74.0	72.666667	C	1
1	2	Japan	69.0	90.0	74.0	53.000000	B	0
3	4	China	47.0	NaN	44.0	30.333333	D	1
4	5	India	76.0	78.0	74.0	51.333333	C	0
5	6	Japan	71.0	83.0	78.0	77.333333	C	1
...	...	...	...	...	...	...	...	...
94	95	USA	79.0	NaN	74.0	26.333333	A	0
95	96	Japan	78.0	81.0	82.0	80.333333	A	0
96	97	USA	65.0	81.0	62.0	64.333333	B	0
97	98	Rassia	63.0	72.0	70.0	68.333333	B	1
98	99	India	58.0	67.0	74.0	41.666667	B	0

98 rows x 8 columns

Conclusion:

In this way we have explored the functions of the python library for Data Identifying and handling the outliers. Data Transformations Techniques are explored with the purpose of creating the new variable and reducing the skewness from datasets.

Group A

Experiment No: 3

Title of the Assignment:

Descriptive Statistics - Measures of Central Tendency and variability

Perform the following operations on any open source dataset (e.g., data.csv)

1. Provide summary statistics (mean, median, minimum, maximum, standard deviation) for a dataset (age, income etc.) with numeric variables grouped by one of the qualitative (categorical) variables. For example, if your categorical variable is age groups and quantitative variable is income, then provide summary statistics of income grouped by the age groups. Create a list that contains a numeric value for each response to the categorical variable.
2. Write a Python program to display some basic statistical details like percentile, mean, standard deviation etc. of the species of 'Iris-setosa', 'Iris-versicolor' and 'Iris- versicolor' of iris.csv dataset.

Provide the codes with outputs and explain everything that you do in this step.

Objective of the Assignment: Students should be able to perform the Statistical operations using Python on any open source dataset.

Prerequisite:

1. Basic of Python Programming
2. Concept of statistics such as mean, median, minimum, maximum, standard deviation etc.

Theory:

1. Summary **statistics**
2. **Summary statistics of income grouped by the age groups**
3. **Display basic statistical details on the iris dataset.**

1. Summary **statistics:**

What is Statistics?

Statistics is the science of collecting data and analysing them to infer proportions (sample) that are representative of the population. In other words, statistics is interpreting data in order to make predictions for the population.

Branches of Statistics:

There are two branches of Statistics.

DESCRIPTIVE STATISTICS: Descriptive Statistics is a statistics or a measure that describes the data.

INFERENTIAL STATISTICS: Using a random sample of data taken from a population to describe and make inferences about the population is called Inferential Statistics.

Descriptive Statistics

Descriptive Statistics is summarising the data at hand through certain numbers like mean, median etc. so as to make the understanding of the data easier. It does not involve any generalisation or inference beyond what is available.

Commonly Used Measures

1. Measures of Central Tendency
2. Measures of Dispersion (or Variability)

Measures of Central Tendency

A Measure of Central Tendency is a one number summary of the data that typically describes the centre of the data.

a. Mean : Mean is defined as the ratio of the sum of all the observations in the data to the total number of observations

Consider the following data points.

17, 16, 21, 18, 15, 17, 21, 19, 11, 23

$$\text{Mean} = \frac{17 + 16 + 21 + 18 + 15 + 17 + 21 + 19 + 11 + 23}{10} = \frac{178}{10} = 17.8$$

b. **Median** : Median is the point which divides the entire data into two equal halves. One-half of the data is less than the median, and the other half is greater than the same. Median is calculated by first arranging the data in either ascending or descending order.

o If the number of observations is odd, the median is given by the middle observation in the sorted form.

o If the number of observations are even, median is given by the mean of the two middle observations in the sorted form.

An important point to note is that the order of the data (ascending or descending) does not affect the median.

To calculate Median, let's arrange the data in ascending order.

11, 15, 16, 17, 17, 18, 19, 21, 21, 23

Since the number of observations is even (10), median is given by the average of the two middle observations (5th and 6th here).

$$\text{Median} = \frac{5^{\text{th}} \text{ Obs} + 6^{\text{th}} \text{ Obs}}{2} = \frac{17 + 18}{2} = 17.5$$

c. **Mode**: Mode is the number which has the maximum frequency in the entire data set, or in other words, mode is the number that appears the maximum number of times. A data can have one or more than one mode.

- If there is only one number that appears the maximum number of times, the data has one mode, and is called Uni-modal.

If there are two numbers that appear the maximum number of times, the data has two modes, and is called Bi-modal.

- If there are more than two numbers that appear the maximum number of times, the data has more than two modes, and is called Multi-modal.

Consider the following data points.

17, 16, 21, 18, 15, 17, 21, 19, 11, 23

Mode is given by the number that occurs the maximum number of times.

Here, 17 and 21 both occur twice. Hence, this is a Bimodal data and the modes are 17 and 21.

- **Measures of Dispersion** (or Variability)

Measures of Dispersion describes the spread of the data around the central value (or the Measures of Central Tendency)

1. **Absolute Deviation from Mean** - The Absolute Deviation from Mean, also called Mean Absolute Deviation (MAD), describes the variation in the data set, in the sense that it tells the average absolute distance of each data point in the set. It is calculated as

$$\text{Mean Absolute Deviation} = \frac{1}{N} \sum_{i=1}^N |X_i - \bar{X}|$$

2. **Variance**-Variance measures how far are data points spread out from the mean. A high variance indicates that data points are spread widely and a small variance indicates that the data points are closer to the mean of the data set. It is calculated as

$$\text{Variance} = \frac{1}{N} \sum_{i=1}^N (X_i - \bar{X})^2$$

3. **Standard Deviation**

The square root of Variance is called the Standard Deviation. It is calculated as

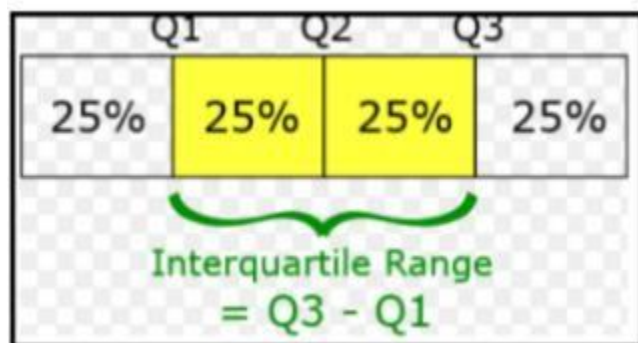
$$\text{Std Deviation} = \sqrt{\text{Variance}} = \sqrt{\frac{1}{N} \sum_{i=1}^N (X_i - \bar{X})^2}$$

4. **Range** - Range is the difference between the Maximum value and the Minimum value in the data set. It is given as

$$\text{Range} = \text{Maximum} - \text{Minimum}$$

5. Quartiles

Quartiles are the points in the data set that divides the data set into four equal parts. Q1, Q2 and Q3 are the first, second and third quartile of the data set.



6. **Skewness** - The measure of asymmetry in a probability distribution is defined by Skewness. It can either be positive, negative or undefined.

$$\text{Skewness} = \frac{3 (\text{Mean} - \text{Median})}{\text{Std Deviation}}$$

Positive Skew - This is the case when the tail on the right side of the curve is bigger than that on the left side. For these distributions, mean is greater than the mode.

Negative Skew - This is the case when the tail on the left side of the curve is bigger than that on the right side. For these distributions, mean is smaller than the mode.

2. Summary statistics of income grouped by the age groups

Problem Statement: For example, if your categorical variable is age groups and quantitative variable is income, then provide summary statistics of income grouped by the age groups. Create a list that contains a numeric value for each response to the categorical variable.

Categorical Variable: Genre

Quantitative Variable: Age

Syntax:

```
df.groupby(['Genre']) ['Age'].mean ()
```

Categorical Variable: Genre

Quantitative Variable: Income

Syntax:

```
df_u=df.rename (columns= {'Annual Income  
k$': 'Income'}, inplace=False)  
(df_u.groupby(['Genre']). Income.mean())
```

To create a list that contains a numeric value for each response to the categorical variable.

```
from sklearn import preprocessing  
enc  
enc_df  
enc_df  
preprocessing. OneHotEncoder()  
pd.DataFrame (enc.fit_transform (df [['Genre']]).toarray())
```

To concat numerical list to dataframe

```
df_encode =df_u.join (enc_df)  
df_encode
```

3. Display basic statistical details on the iris dataset.

Algorithm:

1. Import Pandas Library

2. The dataset is downloaded from UCI repository.

```
csv_url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data'
```

3. Assign Column names

```
col_names
```

```
['Sepal_Length', 'Sepal_Width', 'Petal_Length', 'Petal_Width', 'Species']
```

4. Load Iris.csv into a Pandas data frame

```
iris = pd.read_csv(csv_url, names=col_names)
```

5. Load all rows with Iris-setosa species in variable irisSet

```
irisSet = (iris['Species'] == 'Iris-setosa')
```

6. To display basic statistical details like percentile, mean, standard deviation etc. for

Iris-setosa use describe

```
print('Iris-setosa')
```

```
print(iris[irisSet].describe())
```

7. Load all rows with Iris-versicolor species in variable irisVer

```
irisVer = (iris['Species'] == 'Iris-versicolor')
```

8. To display basic statistical details like percentile, mean, standard deviation etc. for

Iris-versicolor use describe

```
print('Iris-versicolor')
```

```
print(iris[irisVer].describe())
```

9. Load all rows with Iris-virginica species in variable irisVir

```
irisVir = (iris['Species'] == 'Iris-virginica')
```

10. To display basic statistical details like percentile, mean, standard deviation etc. for

Iris-virginica use describe

```
print('Iris-virginica')
```

```
print(iris[irisVir].describe())
```

CONCLUSION:

To summarize, here we discussed how to generate summary statistics using the Pandas library. Here, we discussed how to use pandas methods to generate mean, median, max, min and standard deviation. We also saw describe () method which allows us to generate percentiles, in addition to the mean, median, max, min and standard deviation, for any numerical column. Finally, we showed how to generate aggregate statistics for categorical columns.

Group A

Experiment No 4

Aim:

Create a Linear Regression Model using Python/R to predict home prices using Boston Housing Dataset

Objective:

- To understand linear regression using python
- To do data analysis using linear regression

Prerequisite:

1. **Basic of Python Programming**
2. **Concept of Regression.**

Theory:

Contents for Theory:

1. **Linear Regression : Univariate and Multivariate**
2. **Least Square Method for Linear Regression**
3. **Measuring Performance of Linear Regression**
4. **Training Data set and Testing data set**

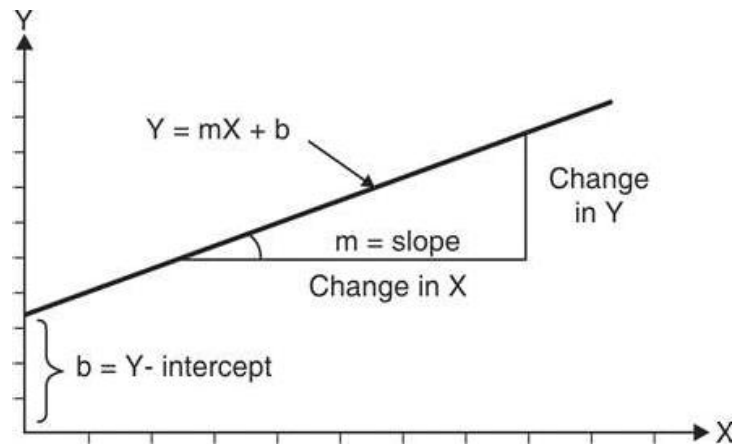
1. **Linear Regression:** It is a machine learning algorithm based on supervised learning. It targets prediction values on the basis of independent variables.

- It is preferred to find out the relationship between forecasting and variables.
- A linear relationship between a dependent variable (X) is continuous; while independent variable(Y) relationship may be continuous or discrete. A linear relationship should be available in between predictor and target variable so known as Linear Regression.

- It is shown as an equation of line like : $Y = m \cdot X + b + e$

Where: b is intercept, m is slope of the line and e is error term.

This equation can be used to predict the value of target variable Y based on given predictor variable(s) X , as shown in Fig. 1.



- Fig. 2 shown below is about the relation between weight (in Kg) and height (in cm), a linear relation. It is an approach of studying in a statistical manner to summarize and learn the relationships among continuous (quantitative) variables.
- Here a variable, denoted by 'x' is considered as the predictor, explanatory, or independent variable.
- Another variable, denoted 'y', is considered as the response, outcome, or dependent variable. While "predictor" and "response" used to refer to these variables.
- Simple linear regression technique concerned with the study of only one predictor variable.

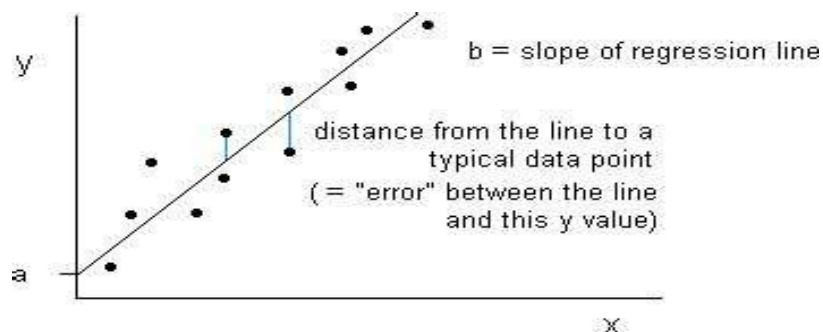


Fig.2: Relation between weight (in Kg) and height (in cm)

MultiVariate Regression: It concerns the study of two or more predictor variables. Usually a transformation of the original features into polynomial features from a given degree is preferred and further Linear Regression is applied on it.

- A simple linear model $Y = a + bX$ in original feature will be transformed into polynomial feature is transformed and further a linear regression applied to it and it will be something like

$$Y = a + bX + cX^2$$

- If a high degree value is used in transformation the curve becomes over-fitted as it captures the noise from data as well.

2. Least Square Method for Linear Regression

- Linear Regression involves establishing linear relationships between dependent and independent variables. Such a relationship is portrayed in the form of an equation also known as the linear model.
- A simple linear model is the one which involves only one dependent and one independent variable. Regression Models are usually denoted in Matrix Notations.
- However, for a simple univariate linear model, it can be denoted by the regression equation

$$\bar{Y} = \beta_0 + \beta_1 X \quad \text{---(1)}$$

where $\hat{y}$ is the dependent or the response variable

x is the independent or the input variable

β_0 is the value of y when $x=0$ or the y intercept

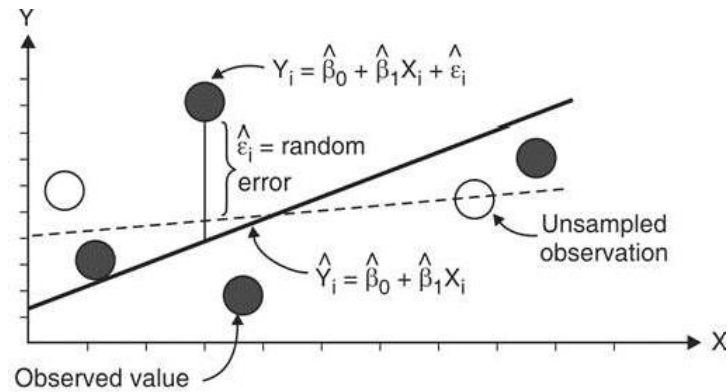
β_1 is the value of slope of the line ϵ is the error or the noise

- This linear equation represents a line also known as the 'regression line'. The least square estimation technique is one of the basic techniques used to guess the values of the parameters and based on a sample set.
- This technique estimates the parameter β_0 and β_1 and by trying to minimize the square of

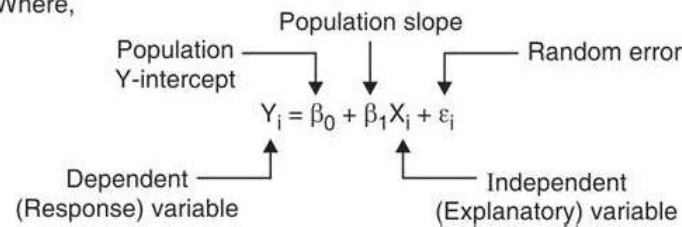
errors at all the points in the sample set. The error is the deviation of the actual sample data point from the regression line. The technique can be represented by the equation.

(2)

$$\min \sum_{i=0}^n (\hat{y} - y)^2$$



Where,



Using differential calculus on equation 1 we can find the values of β_0 and β_1 that the sum of squares (that is equation 2) is minimum.

$$\beta_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} \quad (3)$$

$$\beta_0 = \bar{y} - \beta_1 \bar{x} \quad (4)$$

Once the Linear Model is estimated using equations (3) and (4), we can estimate the value of the dependent variable in the given range only. Going outside the range is called extrapolation which is inaccurate if simple regression techniques are used.

3. Measuring Performance of Linear Regression Mean Square Error:

The Mean squared error (MSE) represents the error of the estimator or predictive model created based on the given set of observations in the sample. Two or more regression models created using a given sample data can be compared based on their MSE. The lesser the MSE, the better the regression model is. When the linear regression model is trained using a given set of observations, the model with the least mean sum of squares error (MSE) is selected as the best model. The Python or R packages select the best-fit model as the model with the lowest MSE or lowest RMSE when training the linear regression models.

Mathematically, the MSE can be calculated as the average sum of the squared difference between the actual value and the predicted or estimated value represented by the regression model (line or plane).

$$MSE = \frac{1}{n} \sum \left(\underbrace{y - \hat{y}}_{\substack{\text{The square of the difference} \\ \text{between actual and} \\ \text{predicted}}} \right)^2$$

An MSE of zero (0) represents the fact that the predictor is a perfect predictor. RMSE:

Root Mean Squared Error method that basically calculates the least-squares error and takes a root of the summed values.

Mathematically speaking, Root Mean Squared Error is the square root of the sum of all errors divided by the total number of values. This is the formula to calculate RMSE

$$\text{RMSE} = \sqrt{\sum_{i=1}^n \frac{1}{n} (\hat{y}_i - y_i)^2}$$

4. Training Data set and Testing data set

- Machine Learning algorithm has two phases
 - Training
 - Testing.
- The input of the training phase is training data, which is passed to any machine learning algorithm and machine learning model is generated as output of the training phase.
- The input of the testing phase is test data, which is passed to the machine learning model and prediction is done to observe the correctness of mode.

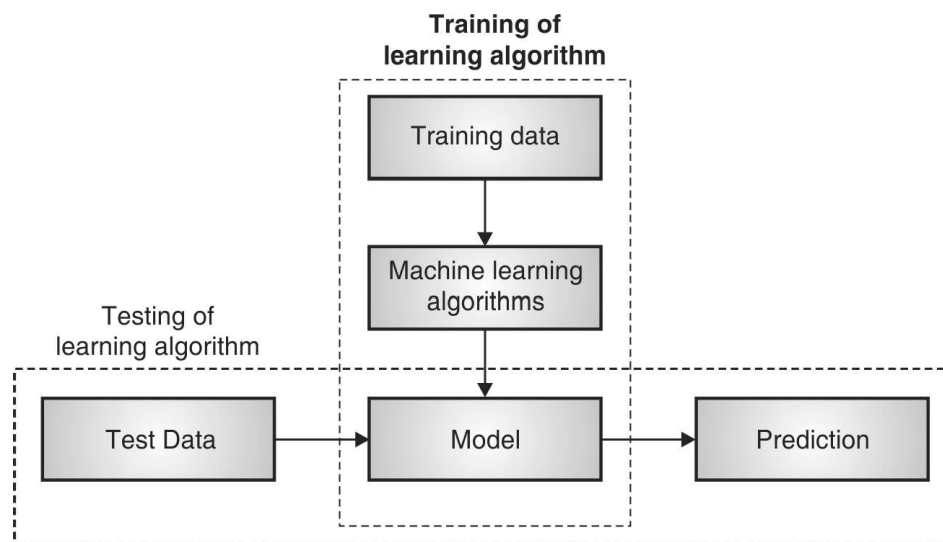


Fig. 1.3.1 : Training and Testing Phase in Machine Learning

(a) Training Phase

- Training dataset is provided as input to this phase.
- Training dataset is a dataset having attributes and class labels and used for training Machine Learning algorithms to prepare models.
- Machines can learn when they observe enough relevant data. Using this one can model algorithms to find relationships, detect patterns, understand complex problems and make decisions.

(b) Testing Phase

- Testing dataset is provided as input to this phase.

Test dataset is a dataset for which class label is unknown. It is tested using model
Training and Testing dataset are completely different.

- A test dataset used for assessment of the finally chosen model.
Testing error is the error that occurs by assessing the model by providing the unknown data to the model.
- In a simple way the actual output of testing data and predicted output of the model does not match the testing error E_{out} is said to have occurred.
- E_{out} is generally observed larger than E_{in} .

(c) Generalization

- Generalization is the prediction of the future based on the past system.
- It needs to generalize beyond the training data to some future data that it might not have seen yet.
- The ultimate aim of the machine learning model is to minimize the generalization error.
- The generalization error is essentially the average error for data the model has never seen.
- In general, the dataset is divided into two partition training and test sets.
- The fit method is called on the training set to build the model.
- This fit method is applied to the model on the test set to estimate the target value and evaluate the model's performance.
- The reason the data is divided into training and test sets is to use the test set to estimate how well the model trained on the training data and how well it would perform on the unseen data.

Algorithm :

Step 1: Import libraries and create alias for Pandas, Numpy and Matplotlib

```
import pandas as pd  
  
import numpy as np  
  
import matplotlib.pyplot as plt  
  
import seaborn as sns  
  
%matplotlib inline
```

Step 2: Import the Boston Housing dataset

```
#import dataset from sklearn
```

```
#from sklearn.datasets import load_boston  
  
#boston_dataset = load_boston()
```

Step 3: Read the Dataset

```
boston=pd.read_csv(r'C:\Users\Student\Documents\HousingData.csv')
```

Step 4: Display information

```
boston
```

Step 5: Find missing Values, if any missing value then fill it by suitable method

```
boston.isnull().sum()
```

Step 6: Find outliers ,if found then Remove it

Step 7: Find Correlation matrix

```
df.corr()
```

```
sns.heatmap(boston.corr(), cmap = 'RdGy')
```

Step 8: Plot the correlation matrix using heat map

```
plt.subplots(figsize=(12,8))  
  
sns.heatmap(boston.corr(), annot=True)
```

Step 9: Divide data into training and testing

```
from sklearn.model_selection import train_test_split  
  
xtrain, xtest, ytrain, ytest = train_test_split(x, y, test_size=0.2, random_state = 0)  
  
print("xtrain shape : ", xtrain.shape)  
  
print("xtest shape : ", xtest.shape)  
  
print("ytrain shape : ", ytrain.shape)  
  
print("ytest shape : ", ytest.shape)
```

Step 10: Select the model and apply Linear Regression on it

```
from sklearn.linear_model import LinearRegression  
  
model = LinearRegression()
```

Step 11: Apply linear regression model on training data

```
from sklearn.linear_model import LinearRegression  
  
model = LinearRegression()
```

Step 12: Plotting Scatter graph to show the prediction

```
from sklearn.metrics import mean_squared_error, mean_absolute_error  
  
mse = mean_squared_error(ytest, y_pred)  
  
mae = mean_absolute_error(ytest, y_pred)  
  
model_score = model.score(xtest, ytest)
```

Step 13: Display mean absolute error, mean square error and model score

```
print("Mean Square Error : ", mse)  
  
print("Mean Absolute Error : ", mae)  
  
print("Model Score : ", model_score)
```

Step 14: Test the model using user input

```
features = np.array([[6, 19]])  
  
prediction = model.predict(features)
```

Step 15: Calculate the R2 Score

```
from sklearn.metrics import r2_score  
  
r2 = r2_score(ytest, y_pred)  
  
r2
```

Conclusion:

In this way we have done data analysis using linear regression for Boston Dataset and predict the price of houses using the features of the Boston Dataset.

